



INSTITUTO POLITÉCNICO NACIONAL



Unidad Profesional Interdisciplinaria en Ingeniería y
Tecnologías Avanzadas

Tarea 3:

fragmentación horizontal

Alumnos:

Gamez Grees Isaac Humberto

Martínez Barrueta Mariana

Zuñiga Áviles Nathan Emiliano

Base De Datos Distribuidas

Fragmentación horizontal

Ejercicio 3: Distribuir la cartera de clientes en los 5 nodos geográficos para que cada región administre y consulte de forma autónoma los datos de sus propios compradores. Tabla **Sales.Customer**, columna **CustomerId** (rangos).

En este ejercicio, la tabla **Sales.Customer** será nuestra **tabla propietaria**, y a partir de ella se realizará la fragmentación horizontal primaria usando el atributo **CustomerId**.

Después se revisaron las tablas que están relacionadas con **Sales.Customer** y que pueden depender de ella para después generar fragmentos derivados. Para identificar estas relaciones se tomaron en cuenta las **llaves foráneas**, ya que estas permiten saber cuándo una tabla depende de otra.

La primera relación es entre **Sales.Customer** y **Sales.SalesOrderHeader**. La tabla **Sales.Customer** tiene como llave primaria **CustomerId**, mientras que **Sales.SalesOrderHeader** tiene como llave primaria **SalesOrderID** y además contiene **CustomerId** como llave foránea.

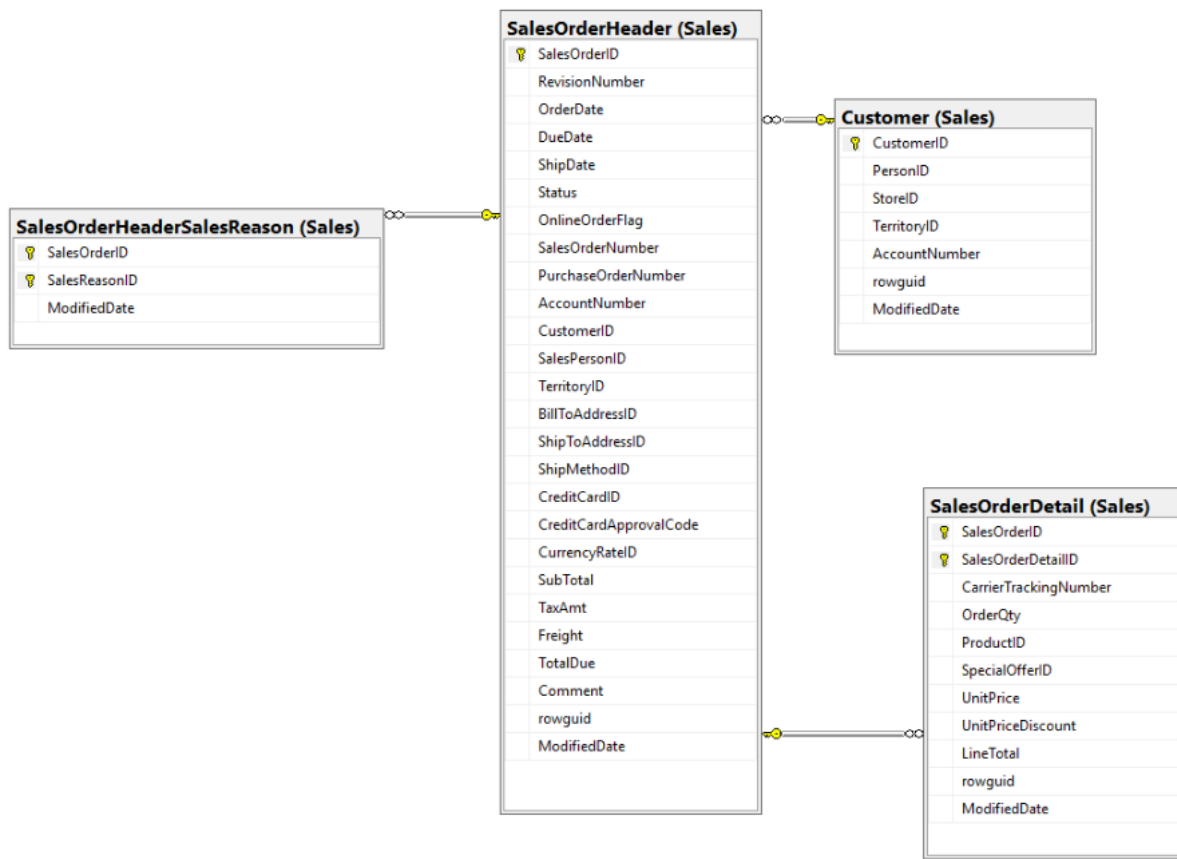
Esto quiere decir que cada orden de venta está relacionada con un cliente, por lo tanto, **Sales.SalesOrderHeader** funciona como **tabla miembro** de **Sales.Customer**. Sin embargo, al mismo tiempo **Sales.SalesOrderHeader** también funciona como **tabla propietaria**, ya que de ella dependen otras tablas como **Sales.SalesOrderDetail** y **Sales.SalesOrderHeaderSalesReason**.

A su vez, **Sales.SalesOrderHeader** se relaciona con **Sales.SalesOrderDetail** mediante **SalesOrderID**. En este caso, **SalesOrderDetail** depende de **SalesOrderHeader**, ya que utiliza **SalesOrderID** como llave foránea para saber a qué orden pertenece cada detalle.

También se encuentra la tabla **Sales.SalesOrderHeaderSalesReason**, la cual se relaciona de la misma manera con **SalesOrderHeader** mediante **SalesOrderID**. Por esta razón, tanto **SalesOrderDetail** como **SalesOrderHeaderSalesReason** funcionan como **tablas miembros** de **SalesOrderHeader**.

De esta forma, el grafo parte de **Sales.Customer**, continúa con **Sales.SalesOrderHeader** y después se divide hacia **Sales.SalesOrderDetail** y **Sales.SalesOrderHeaderSalesReason**.

Grafo relacional:



Para determinar los rangos que se utilizarán en la fragmentación horizontal primaria, primero se realizó un análisis del atributo **CustomerID** de la tabla **Sales.Customer**. Se obtuvo como valor mínimo 1, como valor máximo 30118 y un total de 19820 registros.

```
SELECT
    MIN(CustomerID) AS MinimoID,
    MAX(CustomerID) AS MaximoID,
    COUNT(*) AS TotalClientes
FROM Sales.Customer;
```

	MinimoID	MaximoID	TotalClientes
1	1	30118	19820

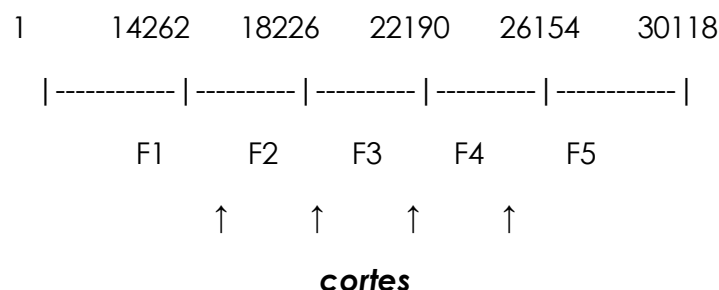
Como se requieren cinco fragmentos, se dividió el total de clientes entre cinco. Por lo tanto, se buscó que cada fragmento estuviera formado inicialmente por 3,964 clientes.

Para encontrar los valores de CustomerID donde se realizarían los cortes, primero se ordenaron los registros por CustomerID y se asignó una posición a cada cliente. De esta manera se pudieron identificar los clientes ubicados en las posiciones 3,964, 7,928, 11,892 y 15,856, que corresponden al final de los primeros cuatro grupos.

```
--Análisis de Sales.Customer
WITH ClientesOrdenados AS
(
    SELECT
        CustomerID,
        ROW_NUMBER() OVER (ORDER BY CustomerID) AS Posicion
    FROM Sales.Customer
),
Grupos AS
(
    SELECT
        CustomerID,
        CASE
            WHEN Posicion <= 3964 THEN 1
            WHEN Posicion <= 7928 THEN 2
            WHEN Posicion <= 11892 THEN 3
            WHEN Posicion <= 15856 THEN 4
            ELSE 5
        END AS Grupo
    FROM ClientesOrdenados
)
SELECT
    Grupo,
    MIN(CustomerID) AS CustomerID_Inicial,
    MAX(CustomerID) AS CustomerID_Final,
    COUNT(*) AS CantidadClientes
FROM Grupos
GROUP BY Grupo
ORDER BY Grupo;
```

	Grupo	CustomerID_Inicial	CustomerID_Final	CantidadClientes
1	1	1	14262	3964
2	2	14263	18226	3964
3	3	18227	22190	3964
4	4	22191	26154	3964
5	5	26155	30118	3964

Por esta razón, el primer fragmento termina en CustomerID = 14262. Aunque este valor sea mayor que 3,964, esto ocurre porque los valores de **CustomerID** no son completamente consecutivos y existen números que no están asignados a ningún cliente.



Posteriormente, se definieron los **predicados simples**, los cuales representan **las condiciones que se aplicaran sobre los datos**. A partir de los puntos de corte obtenidos del análisis anterior, se establecieron los siguientes predicados:

$p1: \text{CustomerID} \leq 14262$

$p2: \text{CustomerID} \leq 18226$

$p3: \text{CustomerID} \leq 22190$

$p4: \text{CustomerID} \leq 26154$

Conjunto de predicados $Pr: \{p1, p2, p3, p4\}$

Una vez definido el conjunto inicial de predicados Pr , se aplicó el algoritmo **COM_MIN**, cuyo objetivo es obtener un conjunto de predicados que sea completo y mínimo.

Un conjunto es **completo** cuando contiene los predicados suficientes para diferenciar los grupos de datos que serán tratados de manera distinta. En nuestro caso, se considera que cada uno de los cinco nodos trabajará con un rango diferente de clientes. Por otro lado, un conjunto es **mínimo** cuando todos los predicados que contiene son necesarios para la fragmentación. Si se elimina uno de nuestros cuatro puntos de corte, dos de los rangos quedarían unidos.

El predicado y su negación permiten separar todos los registros en dos grupos: los que cumplen la condición y los que no la cumplen.

Estas negaciones son necesarias porque, al combinar los predicados, permiten delimitar los distintos rangos que formarán los fragmentos.

Iteración 1:

Se analiza:

$p1: \text{CustomerID} \leq 14262$

Este predicado permite distinguir el primer grupo:

$\text{CustomerID} \leq 14262$

$\text{CustomerID} > 14262$

Por lo tanto, **$P1$** es relevante y se agrega al conjunto: **$Pr' = \{P1\}$**

Iteración 2:

Se analiza:

$P2: \text{CustomerID} \leq 18226$

Al combinarlo con la negación de **$p1$** , se puede distinguir un nuevo rango:

$\neg P1 \wedge P2$

Equivale a:

$14262 < \text{CustomerID} \leq 18226$

Por lo tanto, **$P2$** es relevante y se agrega al conjunto: **$Pr' = \{P1, P2\}$**

Iteración 3:

Con:

$$P3: \text{CustomerID} \leq 22190$$

Se obtiene un nuevo rango:

$$\neg P2 \wedge P3$$

Equivale a:

$$18226 < \text{CustomerID} \leq 22190$$

Por lo tanto, **P3** es relevante y se agrega al conjunto: $Pr'=\{P1,P2,P3\}$

Iteración 4:

Finalmente:

$$P4: \text{CustomerID} \leq 26154$$

Permite distinguir:

$$\neg P3 \wedge P4$$

Equivalente a:

$$22190 < \text{CustomerID} \leq 26154$$

Mientras que los registros posteriores al último corte cumplen:

$$\neg P4$$

Es decir:

$$\text{CustomerID} > 26154$$

Debido a que los cuatro predicados son necesarios para conservar los cinco rangos definidos, ninguno es eliminado. Por lo tanto, el conjunto completo y mínimo queda:

$$Pr'=\{P1,P2,P3,P4\}$$

Donde:

$$P1: \text{CustomerID} \leq 14262$$

$$P2: \text{CustomerID} \leq 18226$$

$$P3: \text{CustomerID} \leq 22190$$

$$P4: \text{CustomerID} \leq 26154$$

Una vez obtenido el conjunto completo y mínimo de predicados mediante COM_MIN, se procedió a generar los **minitérminos**. Estos permiten combinar los predicados simples y sus negaciones para describir de forma exacta cada uno de los rangos que posteriormente formarán los fragmentos.

En este caso, los predicados simples representan los límites superiores de cada rango. Para establecer también el límite inferior se utiliza la negación del predicado anterior.

Minitérmino 1:

El primer rango comienza desde el inicio de nuestros datos, así que únicamente necesitamos:

$$m1 = p1$$

Por lo tanto:

$$m1: CustomerID \leq 14262$$

Minitérmino 2:

$$m2 = \neg P1 \wedge P2$$

Equivalente:

$$m2 = 14262 < CustomerID \leq 18226$$

Minitérmino 3:

$$\neg P2 \wedge P3$$

Como:

$$\neg P2 = CustomerID \leq 18226$$

Y:

$$P3 = CustomerID \leq 22190$$

Entonces:

$$m3 = 18226 < CustomerID \leq 22190$$

Minitérmino 4:

$$m4 = \neg P3 \wedge P4$$

Por lo tanto:

$$m4 = 22190 < CustomerID \leq 26154$$

Minitérmino 5:

Después del último punto de corte ya no existe otro predicado superior. Por eso simplemente tomamos la negación de **P4**:

$$m5 = \neg P4$$

Como:

$$P4: \text{CustomerID} \leq 26154$$

Entonces:

$$m5: \text{CustomerID} > 26154$$

Y finalmente tenemos el conjunto $M=\{m1,m2,m3,m4,m5\}$

De esta manera, aunque se utilizaron cuatro predicados simples, se obtuvieron cinco minitérminos, ya que los cuatro puntos de corte permiten dividir los datos en cinco rangos diferentes.

Una vez definido el conjunto de minitérminos M , cada minitérmino se utiliza como condición para obtener uno de los fragmentos horizontales primarios de la tabla **Sales.Customer**.

Para representar esta operación se utiliza el símbolo de selección σ , el cual indica que se tomarán de la tabla únicamente los registros que cumplan con la condición indicada por cada minitérmino.

Primer fragmento:

Tenemos:

$$m1=P1$$

Y:

$$P1= \text{CustomerID} \leq 14262$$

Por lo tanto:

$$\text{Customer1} = \sigma_{\text{CustomerID} \leq 14262} (\text{Sales.Customer})$$

El fragmento **Customer1** contiene todos los clientes de **Sales.Customer** cuyo **CustomerID** sea menor o igual a 14262.

Segundo fragmento:

Tenemos:

$$m2= \neg P1 \wedge P2$$

Que ya sabemos que es equivalente a:

$$14262 < \text{CustomerID} \leq 18226$$

Por lo tanto:

$$\text{Customer2} = \sigma_{14262 < \text{CustomerID} \leq 18226} (\text{Sales.Customer})$$

Tercer fragmento:

$Customer3 = \sigma_{18226 < CustomerID \leq 22190} (Sales.Customer)$

Cuarto fragmento:

$Customer4 = \sigma_{22190 < CustomerID \leq 26154} (Sales.Customer)$

Quinto fragmento:

Finalmente:

$Customer5 = \sigma_{CustomerID > 26154} (Sales.Customer)$

Por lo tanto, los fragmentos primarios son:

$Customer1 = \sigma_{CustomerID \leq 14262} (Sales.Customer)$

$Customer2 = \sigma_{14262 < CustomerID \leq 18226} (Sales.Customer)$

$Customer4 = \sigma_{22190 < CustomerID \leq 26154} (Sales.Customer)$

$Customer4 = \sigma_{22190 < CustomerID \leq 26154} (Sales.Customer)$

$Customer5 = \sigma_{CustomerID > 26154} (Sales.Customer)$

Una vez obtenidos los cinco fragmentos primarios de **Sales.Customer**, se procedió a generar los fragmentos derivados de la tabla miembro **Sales.SalesOrderHeader**.

A diferencia de la fragmentación primaria, en este caso no se definieron nuevos rangos sobre el atributo **SalesOrderID**. Los fragmentos de **SalesOrderHeader** se obtuvieron a partir de los fragmentos previamente generados en **Sales.Customer**.

Para ello se utilizó la operación **SEMI-JOIN**, tomando como atributo de relación **CustomerID**. De esta manera, cada fragmento de **SalesOrderHeader** contiene únicamente las órdenes asociadas con los clientes pertenecientes al fragmento correspondiente de **Sales.Customer**, es decir, que las órdenes de cada cliente **sigan al fragmento donde quedó su cliente**.

El **SEMI-JOIN** nos ayuda a obtener los registros de una tabla miembro que corresponden a un fragmento de la tabla propietaria, es decir, que queremos seleccionar de una tabla solamente las filas que tengan relación con las filas de otra por medio de un atributo. El símbolo que lo representa es: $\bowtie$

La condición que relaciona las tablas es:

$SalesOrderHeader.CustomerID = Customer.CustomerID$

Por lo tanto:

$SOH1 = SalesOrderHeader \bowtie Customer1$

$SOH2 = SalesOrderHeader \bowtie Customer1$

$SOH3 = SalesOrderHeader \bowtie Customer1$

SOH4=SalesOrderHeader ⋈ Customer1

SOH5=SalesOrderHeader ⋈ Customer1

Una vez generados los fragmentos derivados de **Sales.SalesOrderHeader**, estos pueden utilizarse como base para fragmentar la tabla **Sales.SalesOrderDetail**.

Esta tabla se encuentra relacionada con **SalesOrderHeader** mediante el atributo **SalesOrderID**. Por lo tanto, cada fragmento de **SalesOrderDetail** deberá contener únicamente los detalles correspondientes a las órdenes del fragmento de **SalesOrderHeader** asociado.

La condición que relaciona las tablas es:

SalesOrderDetail.SalesOrderID= SalesOrderHeader.SalesOrderID

Y los fragmentos son:

SOD1=SalesOrderDetail ⋈ SOH1

SOD2= SalesOrderDetail ⋈ SOH2

SOD3= SalesOrderDetail ⋈ SOH3

SOD4= SalesOrderDetail ⋈ SOH4

SOD5= SalesOrderDetail ⋈ SOH5

Si una orden pertenece a SOH_2, entonces los detalles de esa orden deben quedar en SOD_2.

De forma similar, **Sales.SalesOrderHeaderSalesReason** también se fragmenta de manera derivada a partir de los fragmentos de **SalesOrderHeader**, debido a que ambas tablas se encuentran relacionadas mediante **SalesOrderID**.

Cada fragmento contendrá únicamente los motivos de venta relacionados con las órdenes pertenecientes al fragmento correspondiente de **SalesOrderHeader**.

La condición que relaciona las tablas es:

SalesOrderHeaderSalesReason.SalesOrderID= SalesOrderHeader.SalesOrderID

Entonces:

SOHSR1= SalesOrderHeaderSalesReason ⋈ SOH1

SOHSR2= SalesOrderHeaderSalesReason ⋈ SOH2

SOHSR3= SalesOrderHeaderSalesReason ⋈ SOH3

SOHSR4= SalesOrderHeaderSalesReason ⋈ SOH4

SOHSR5= SalesOrderHeaderSalesReason ⋈ SOH5

Una vez generados los fragmentos primarios y derivados, se verificó que el diseño cumpliera con las tres reglas de fragmentación: completitud, reconstrucción y disyunción.

Compleitud:

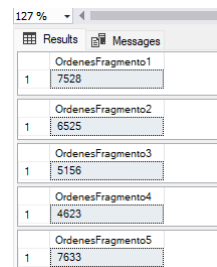
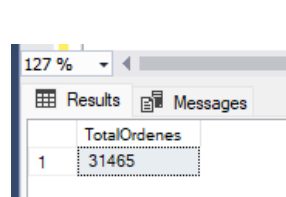
Esta regla establece que todos los registros de la relación original deben encontrarse incluidos en alguno de los fragmentos. En **Sales.Customer** se tienen 19,820 registros y cada uno de los cinco fragmentos contiene 3,964 clientes, por lo que:

$$3964+3964+3964+3964+3964=19820$$

De esta manera, ningún cliente queda fuera del diseño.

Para Sales.SalesOrderHeader, la suma de los registros de los cinco fragmentos derivados es:

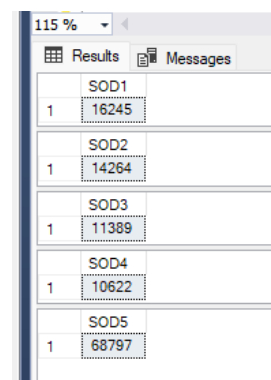
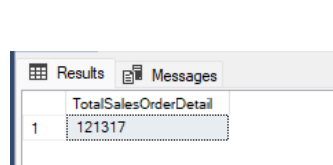
$$7528+6525+5156+4623+7633=31465$$



Fragmento	Registros
OrdenesFragmento1	7528
OrdenesFragmento2	6525
OrdenesFragmento3	5156
OrdenesFragmento4	4623
OrdenesFragmento5	7633

Para Sales.SalesOrderDetail, la suma de los registros de los cinco fragmentos derivados es:

$$16245+14264+11389+10622+68797=121317$$



Fragmento	Registros
SOD1	16245
SOD2	14264
SOD3	11389
SOD4	10622
SOD5	68797

Para **Sales.SalesOrderHeaderSalesReason**, la suma de los registros de los cinco fragmentos derivados es:

$$7871+6444+5096+4365+3871=27647$$

	TotalSalesReason
1	27647

	SOHSR1
1	7871

	SOHSR2
1	6444

	SOHSR3
1	5096

	SOHSR4
1	4365

	SOHSR5
1	3871

Reconstrucción:

Esta regla establece que debe ser posible reconstruir la relación original a partir de sus fragmentos mediante una **unión**.

Para **Sales.Customer**:

$Sales.Customer = Customer1 \cup Customer2 \cup Customer3 \cup Customer4 \cup Customer5$

Si volvemos a juntar los cinco fragmentos, debemos obtener nuevamente los 19,820 clientes originales. Entonces **$3964(5)=19820$** obtenemos nuevamente la tabla original.

Para **SalesOrderHeader**:

$SalesOrderHeader = SOH1 \cup SOH2 \cup SOH3 \cup SOH4 \cup SOH5$

$$SOH_1 = 7528 + SOH_2 = 6525 + SOH_3 = 5156 + SOH_4 = 4623 + SOH_5 = 7633 = 31465$$

Y nuevamente obtenemos la tabla original.

Para **SalesOrderDetail**:

$SalesOrderDetail = SOH1 \cup SOH2 \cup SOH3 \cup SOH4 \cup SOH5$

Para **SalesHeaderSalesReason**:

$SalesOrderDetail = SOHSR1 \cup SOHSR2 \cup SOHSR3 \cup SOHSR4 \cup SOHSR5$

De la misma forma, al unir los cinco fragmentos derivados de **SalesOrderDetail** se debe recuperar nuevamente la tabla **Sales.SalesOrderDetail**. Esto mismo ocurre con **SalesOrderHeaderSalesReason**, ya que la unión de sus cinco fragmentos debe producir todos los registros de la relación original.

Disyunción:

Esta regla establece que un registro no debe pertenecer simultáneamente a dos fragmentos horizontales diferentes. Es decir, queremos evitar duplicados entre fragmentos.

Por ejemplo, para **SalesCustomer** tenemos CustomerID = 15000, el cual esta únicamente en el fragmento F2 $\rightarrow > 14262$ y ≤ 18226 y pertenece a Customer2.

Además, para **SalesOrderHeader** se conserva exactamente igual porque cada orden tiene un **CustomerID** y ese cliente solamente pertenece a **un fragmento** de **Customer**. Por ejemplo, tenemos CustomerID = 15000 $\rightarrow$ Customer2 $\rightarrow$ todas sus órdenes $\rightarrow$ SalesorderHeader2.

Con base en estas comprobaciones, el diseño propuesto cumple con las tres reglas requeridas para una fragmentación horizontal correcta.

A partir de este fragmento, la misma lógica continúa con **SalesOrderDetail**, ya que cada detalle pertenece a una orden mediante **SalesOrderID**. Por lo tanto, si una orden pertenece a **SalesOrderHeader2**, todos los detalles relacionados con esa orden pertenecerán únicamente a **SalesOrderDetail2**.

La cadena queda de la siguiente forma: **CustomerID = 15000 $\rightarrow$ Customer2 $\rightarrow$ SalesOrderHeader2 $\rightarrow$ SalesOrderDetail2**

De la misma manera, **SalesOrderHeaderSalesReason** también depende de **SalesOrderHeader** mediante **SalesOrderID**. Por lo tanto, si una orden pertenece a **SalesOrderHeader2**, los motivos relacionados con esa orden pertenecerán únicamente a **SalesOrderHeaderSalesReason2**.

Entonces la cadena completa puede representarse como: **CustomerID = 15000 $\rightarrow$ Customer2 $\rightarrow$ SalesOrderHeader2 $\rightarrow$ SalesOrderDetail2**

Y también: **CustomerID = 15000 $\rightarrow$ Customer2 $\rightarrow$ SalesOrderHeader2 $\rightarrow$ SalesOrderHeaderSalesReason2**

Conclusiones:

Cada uno de los fragmentos generados puede ser asignado a una región diferente para que cada nodo administre la información que le corresponde. Aunque los rangos se definieron para que al inicio hubiera la misma cantidad de clientes en cada grupo, con el tiempo esto podría cambiar si se agregan nuevos registros, por lo que sería necesario revisar nuevamente los rangos para mantener una distribución equilibrada.

También se pudo observar que la fragmentación no solamente se queda en la tabla Sales.Customer, sino que puede continuar hacia las tablas relacionadas. En este caso, los fragmentos de Sales.Customer permiten obtener los fragmentos de

SalesOrderHeader y, a partir de estos, también los de SalesOrderDetail y SalesOrderHeaderSalesReason, manteniendo la relación entre los datos.

En general, el diseño permitió entender cómo una tabla principal puede dividirse en varios fragmentos y cómo esa misma división puede propagarse hacia otras tablas relacionadas mediante sus llaves foráneas. Además, al comprobar las reglas de completitud, reconstrucción y disyunción, se verificó que los datos pueden dividirse sin perder registros ni generar duplicados entre los fragmentos.